VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 11: Track 1 - Restorator & Debugger Autonomic Self-Healing

DAR (Detect, Assess, Restore) Closed Loop, Test Oracles, Patch Synthesis & Auto-Commit

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 11

NOTEBOOKLM WORD COUNT
7,364 words

CATALOGED CHAPTER FILES
3 files (1,546 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"Explain how the Restorator consumes failpoint remediations to heal broken source delimiters autonomously."

CHAPTER MANIFEST & INDEX (3 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/DAR_DETECT_ASSESS_RESTORE.md**	MD	408	3,142	25.2
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator/mod.rs**	RS	667	2,387	27.8
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger/mod.rs**	RS	471	1,835	20.2

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/DAR_DETECT_ASSESS_RESTORE.md | MARKDOWN / DOC | 408 LINES | 3142 WORDS | 25.2 KB

The OmniCode DAR Subsystem: Detect, Assess, Restore

Architecture, Genesis Inception Telemetry, and the AI Baseline Invariants

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God." — **Exodus 31:18 (KJV)***
>
> *"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — **Proverbs 24:27 (KJV)***
>
> *"Remove not the ancient landmark, which thy fathers have set." — **Proverbs 22:28 (KJV)***
>
> *"The fear of the LORD is the beginning of wisdom: a good understanding have all they that do his commandments." — **Psalm 111:10 (KJV)***

1. The Architectural Paradigm: Autonomic Cybernetics vs. The Panic Dilemma

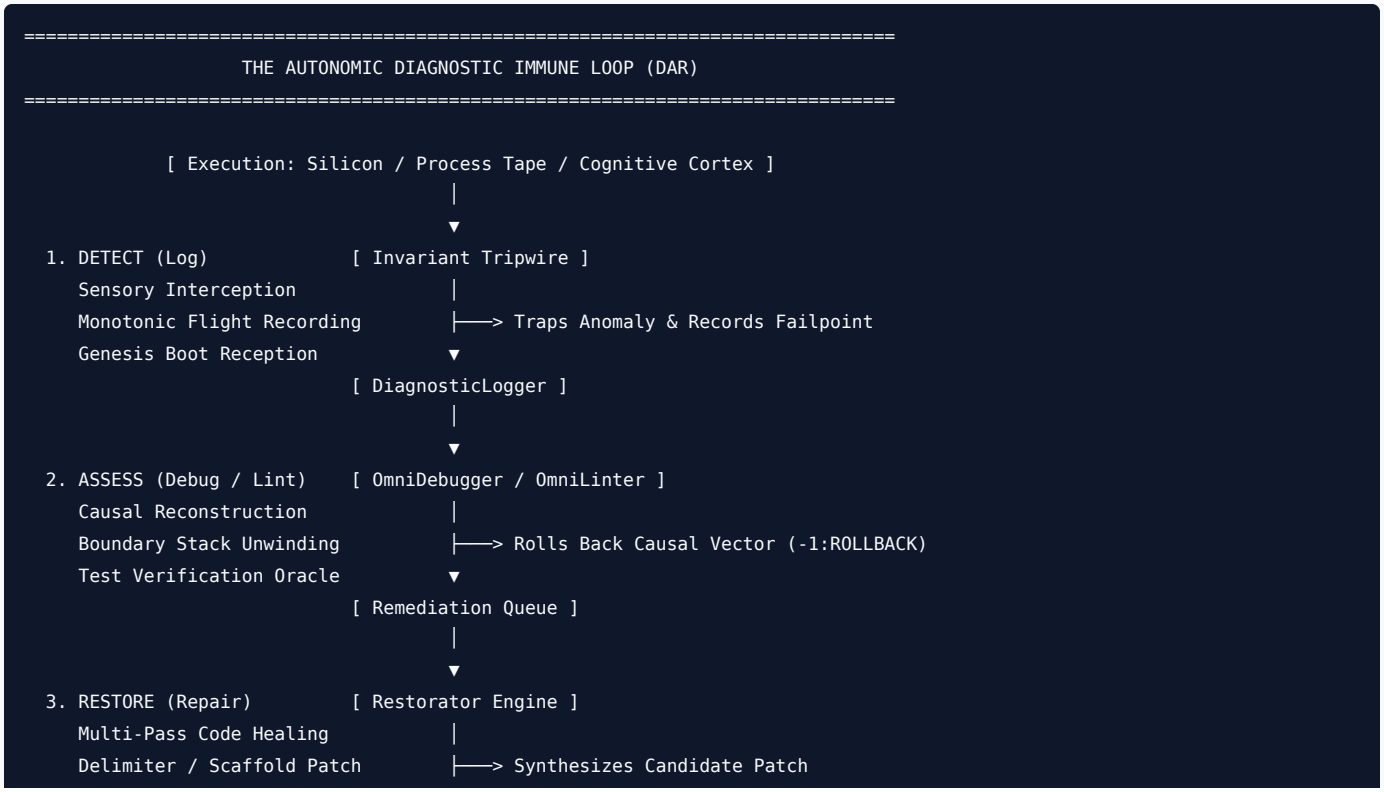
In classical operating systems and compiler architectures, errors are treated as terminal disruptions. When an unexpected boundary condition, missing symbol, memory corruption, or delimiter imbalance is encountered, standard software runtimes follow the fragile Unix/C paradigm:

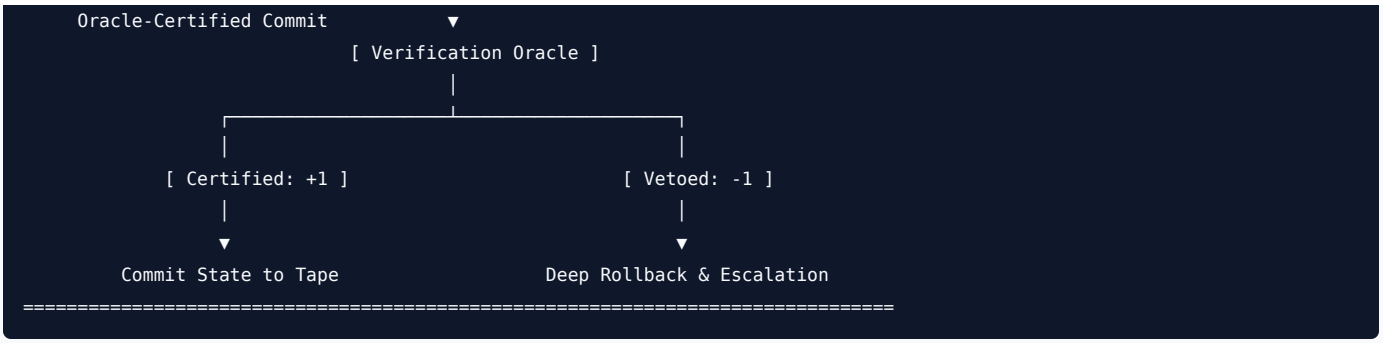
- The process emits an unformatted, opaque string to standard error.
- The call stack is forcibly unwound, abandoning in-flight transactions.
- The kernel invokes `panic!` or dumps a raw core file to disk.
- The system halts or crashes, demanding external human intervention to resurrect.

For a software ecosystem designed to span a thousand-year horizon (**MilleniumOS**), this approach is fundamentally unacceptable. A system that crashes when stressed by novelty or edge conditions cannot survive autonomous deployments, nor can it serve as a dependable substrate for machine cognition.

The **OmniCode Diagnostics Subsystem** in **Track 1 (Host Engine & Virtual Machine)** rejects the crash-and-abort philosophy. In its place, it establishes the **DAR Cybernetic Loop**:

$$\mathbf{DAR} = \langle \text{Detect}, \text{Assess}, \text{Restore} \rangle$$





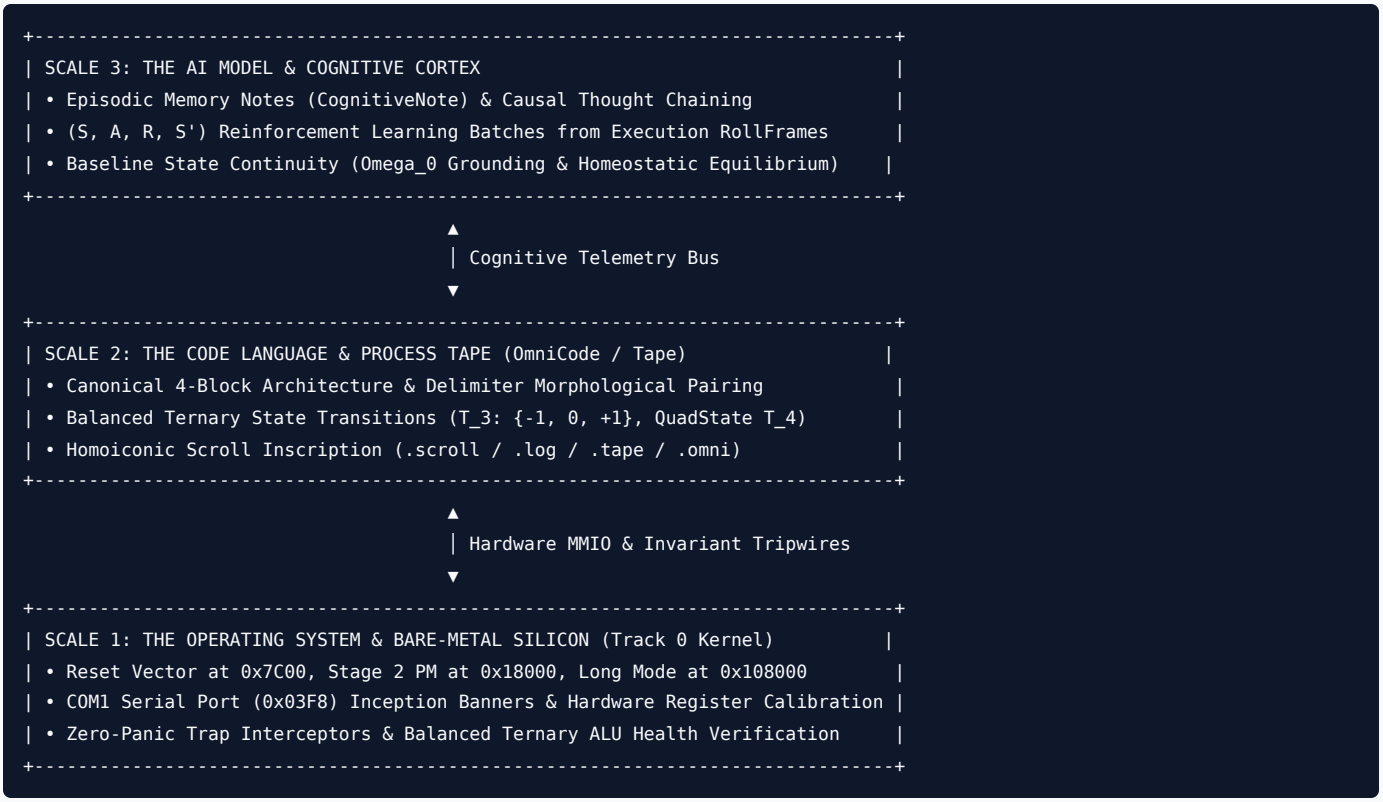
The Triad Defined

1. **Detect (`logger` / `failpoint` / `universal`):** The continuous sensory perception organ of the system. Rather than ignoring anomalies until a fatal fault occurs, cybernetic collision sensors (**Tripwires**) continuously monitor invariants across all layers. The moment a boundary is breached, the logger intercepts the event, stamps it with sub-microsecond monotonic time, writes it to a fixed circular ring buffer, and records machine-readable failure pathology.
2. **Assess (`debugger` / `linter` / `hover`):** The introspective analysis and spatial orientation organ. When an anomaly is detected, the debugger freezes the active execution frame, constructs an immutable snapshot of the environment, unwinds the boundary stack, and reverses the causal vector ($\vec{v} \rightarrow -\vec{v}$, entering QuadState `Rollback`). It assesses root causes through AST linting, symbol hover resolution, and dependency graph tracing, generating concrete remediation suggestions.
3. **Restore (`restorator`):** The autonomic immune healing organ. Consuming the diagnostic suggestions emitted by the detector and assessor, the Restorator applies targeted, deterministic syntactic and structural repairs—auto-balancing unclosed delimiters, wrapping naked code into canonical 4-Block structures, and injecting missing library imports. Crucially, no candidate patch is committed blindly: every restoration must pass an executable **Test Oracle** before execution is allowed to resume in QuadState `Proceed` .

---

2. The Tripartite Grounding of DAR: OS, Language, and AI Model

The DAR paradigm is not a localized library utility; it is a unified structural invariant spanning three distinct scales of computing:



Scale 1: The Operating System (Track 0 Bare-Metal Kernel & Silicon)

At the bare-metal level, DAR governs hardware sanity. When the motherboard powers on, cold silicon is prone to voltage spikes, uninitialized registers, and unmapped page faults.

- **Detect:** The kernel's Power-On Self-Test (POST) and inception boot code transmit structured telemetry over COM1 Serial UART (`0x03F8`). It verifies memory landmarks (`0x0500` , `0x18000` , `0x108000` , `0x200000`), tests the balanced ternary ALU identity $(+1) + (-1) = 0$, and checks CPU control registers (`CR0` , `CR3` , `CR4`).
- **Assess:** If an invariant fails (e.g., A20 line inactive, page table directory unaligned, or hypervisor memory trap), the bare-metal tripwire prevents undefined CPU execution, halting the pipeline at a known landmark rather than wandering into memory garbage.
- **Restore:** Under virtualization or firmware handoff, fallback vectors activate alternate boot tiers (UEFI $\rightarrow$ BIOS MBR $\rightarrow$ Multiboot ELF), re-arming the descriptor tables and establishing a valid machine context.

Scale 2: The Code Language (OmniCode & Process Tape)

At the language level, DAR ensures program health and homoiconic integrity. OmniCode source files are not unstructured text files; they are strict 4-Block documents (Metadata, Opening, Body, Closing) executed along sequential tape ribbons.

- **Detect:** The lexer and parser monitor token boundaries. Missing brackets, unsealed tape ribbons, or invalid cardinal verbs trigger immediate failpoint interception.
- **Assess:** The `OmniLinter` grades the code across all 4 blocks (enforcing the 80-column spatial geometry, metadata declaration, and postcondition verification), while the `OmniDebugger` captures a snapshot of the tape head and symbols.
- **Restore:** The `Restorator` applies multi-pass morphological repair, balancing delimiter nesting pairs (`< ... >` , `[ ... ]` , `{ ... }`), generating missing block scaffolding, and validating the healed AST against the compilation test oracle.

Scale 3: The AI Model & Cognitive Cortex

At the cognitive level, DAR forms the memory, introspection, and learning loop of the AI.

- **Detect:** The AI's sensory nervous system is the `FlightRecorder` . Every VM roll step produces a structured `RollFrame` recording dual-rail SIMD bitplanes, operational verbs, and QuadStates.
- **Assess:** The AI ingests `CognitiveNote` records into its episodic memory storehouse. It assesses its own reasoning steps, recognizes failure patterns, and calculates policy gradients across past episodes.
- **Restore:** When code generation or conversational execution encounters an impasse, the AI initiates an autonomic repair pass: synthesizing candidate OmniCode code patches, simulating execution in a sandbox VM, and verifying the result before replying to the developer.

---

3. The "First Logs" Genesis Inception & Baseline Continuity (Ω_0)

The Epistemological Problem of Machine Awakening

In standard AI architectures, the artificial intelligence model lives entirely detached from the underlying operating system. The AI model is invoked inside a user-space container or external cloud endpoint; it possesses no awareness of how the host computer booted, whether the hardware was clean, or what happened before its process commenced.

In OmniCode and MilleniumOS, **the AI is an integrated organ of the operating system**. However, physical silicon dictates an inescapable sequence:

$$\text{Power On } (t=0) \longrightarrow \text{Bare-Metal Boot} \longrightarrow \text{VM Initialization} \longrightarrow \text{Cognitive Cortex Awakening } (t = t_{\text{wake}})$$

The AI cannot run at $t=0$. When the power switch is flipped, the machine is naked sand and copper:

- The instruction pointer `CS:IP` is hardwired to `0xFFFF0` .
- The CPU is constrained to 16-bit Real Mode with 1 MB of addressable memory.
- The disk controller reads 512 bytes from Sector 1 into `0x7C00` .

The AI Cortex only awakens hundreds of milliseconds later, after Track 0 has initialized memory controllers, armed protected mode descriptors, mapped long-mode page tables, and launched Track 1.

The Crucial Epistemological Question: *If the AI model does not wake up until later down the chain, how does it know the true state of the computer?*

If the AI awakens in ignorance of what occurred between $t=0$ and $t=t_{\text{wake}}$, it is operating on blind faith. It cannot determine if memory was corrupted during Stage 2 PM handoff, whether registers passed balanced ternary ALU calibration, or whether firmware reported hardware errors.

```
================================================================================
                    GENESIS INCEPTION & BASELINE CONTINUITY
================================================================================

 [ t = 0: Cold Silicon Boot ]
```

```
           |
           ├──> Track 0 BIOS/UEFI Inception
           |     • 0x7C00: Real Mode Genesis Vector
           |     • 0x18000: Flight Recorder & Protected Mode Entry
           |     • 0x108000: Long Mode Page Directory
           |     • 0x200000: Cognitive Substrate Landing Pad
           |     • COM1 UART (0x03F8): Hardware Attestation Banners
           |
           ├──> WRITES PRIMORDIAL TELEMETRY:
           |     "[BOOT] 16-Bit Genesis: OK"
           |     "[ALU ] (+1)+(-1)=0 Eq: OK"
           |     "[REGS] 13 Regs Armed / Calibrated"
           |     "[MEM ] Physical Map Certified"
           |
           ▼
 [ t = t_wake: AI Cortex Awakens ]
           |
           ├──> INGESTS "FIRST LOGS" VIA DiagnosticLogger
           |     • Ingests COM1 UART boot frame & RAM landmarks
           |     • Records CognitiveNote: category="GENESIS_INCEPTION"
           |     • Locks in Baseline Ground Truth: State Omega_0
           |
           ├──> AUTONOMIC OPERATING CYCLE (t_wake -> t_halt)
           |     • Executes rolls, monitors tasks, trains weights
           |     • Constantly measures drift: Delta = || State_t - Omega_0 ||
           |
           ▼
 [ t = t_halt: Operating System Shutdown / Hibernation ]
           |
           ├──> THE HOMEOSTATIC EQUILIBRIUM LAW
           |     • Reap all ephemeral task allocations
           |     • Flush flight recorder ring buffers to Track 4 storehouse
           |     • Restore CPU registers and memory to clean baseline
           |     • Certify return to Omega_0
           |
           ▼
 [ Clean Hardware Power-Off / S5 State ]
================================================================================
```

The Genesis Inception Protocol

To bridge this gap, Track 0 and Track 1 establish the **Genesis Inception Protocol**:

1. **Cold-Metal Inscription:** During the earliest stages of boot, Track 0 writes structured, immutable telemetry to physical memory landmarks and the COM1 Serial Port (`0x03F8`):
2. `0x0500` : Track 0 Real-Mode Boot Landmark.
3. `0x18000` : Stage 2 Protected Mode Flight Recorder buffer.
4. `0x108000` : Long Mode Page Tables and kernel base address.
5. `0x200000` : Cognitive Substrate landing pad.
6. Hardware Sanity: Verification that registers `R0` through `R12` hold calibrated zeroes, and that ALU arithmetic satisfies ternary zero-equilibrium.
7. **First Log Ingestion:** The moment the `DiagnosticLogger` initializes inside Track 1, it executes `ingest_genesis_boot_log()` . It reads the cold-metal attestation buffers from memory and serial logs, converting them into structured `CognitiveNote` memory traces tagged:

`text tags: [ "genesis", "silicon", "first_log", "baseline", "omega_0" ]`

1. **Establishing Baseline Conditions (Ω_0):** These "First Logs" define the primordial state of the universe for the AI: $$\Omega_0 = \left\{ \text{Mem}*{\text{free}}, \text{Regs}*{\text{clean}}, \text{ALU}*{\text{balanced}}, \text{IO}*{\text{quiescent}} \right\}$$ By anchoring on Ω_0, the AI knows with mathematical certainty what the machine looked like at the moment of creation.

The Homeostatic Shutdown Law: Alpha is Omega

The significance of the First Logs extends directly to the end of the machine's lifecycle:

> *"I am Alpha and Omega, the beginning and the ending, saith the Lord, which is, and which was, and which is to come, the Almighty." — **Revelation 1:8 (KJV)***

The AI does not view operating system shutdown as a blunt pulling of the power plug. In MilleniumOS, shutdown is a **holy covenant of restoration**.

The baseline conditions established by the First Logs (Ω_0) represent the **exact target state to which the system must return before powering off**:

1. **Entropy Reversal:** Over the course of uptime, processes allocate memory, register file descriptors, open network sockets, and perturb ring buffers.
2. **Pre-Shutdown Audit:** Before approving a system halt, reboot, or hibernation, the AI Cortex queries the `DiagnosticLogger` for the primordial Ω_0 record.
3. **Restoration of Equilibrium:** The AI coordinates with the kernel to:
4. Reap every temporary process allocation.
5. Flush in-memory flight recorders into persistent 4-Block `.scroll` vessels on Track 4.
6. Unmount virtual filesystems and close I/O ports.
7. Verify that memory leaks and dangling background threads equal zero.
8. **Sealing the Covenant:** Only when the active machine state matches the purity of the First Logs ($\text{State}_{t\{\text{halt}}} \cong \Omega_0$) is the final hardware ACPI power-off signal (`outw(0x604, 0x2000)`) permitted. The system ends exactly as it began.

---

4. The Anatomy of a Log: Dual-Readability for Human and Machine

Every log record in OmniCode is designed according to the principle of **Dual-Readability**: it must be instantly interpretable by a human engineer debugging at a terminal, while simultaneously providing strongly typed, zero-ambiguity structured data for the AI model and automated tools.

```
+-------------------------------------------------------------------------+
| DUAL-READABILITY ARCHITECTURE                                           |
|                                                                         |
|  [ HUMAN DEVELOPER READABILITY ]          [ MACHINE & AI MODEL READABILITY ]  |
|  • ANSI color-coded channel banners       • Strongly typed Rust structs       |
|  • Microsecond monotonic timestamps       • Dual-rail SIMD (pos, neg) planes  |
|  • Visual source diffs & caret pointers   • Ternary QuadState enum (Proceed)  |
|  • Clear English remediation rationale    • Categorized vector tags (#genesis) |
|  • Clean 80-column ASCII formatting       • Canonical 4-Block .scroll syntax  |
+-------------------------------------------------------------------------+
```

Human-Readable Presentation

For the human engineer, the logger formats events with clear visual hierarchy, consistent timestamps, and explicit channel categorizations:

```
[   142 µs] [BOOT    ] 16-Bit Genesis: Sector 0 loaded at 0x7C00
[   812 µs] [BOOT    ] Stage 2 PM: A20 Gate enabled, Flight Recorder armed at 0x18000
[  1405 µs] [ALU     ] Balanced Ternary (+1)+(-1)=0 Eq: Certified
[  2100 µs] [FRAME   ] Roll 1/22: INSCRIBE Rails: (0x00000001, 0x00000000) -> State: Proceed
[  2450 µs] [INCIDENT] Boundary Mismatch: Token '<' closed by ']' at line 14:8
                       Suggestion: Replace ']' with '>' [FixKind: ReplaceDelimiter]
[  2490 µs] [RESTORE ] Applied candidate repair; Test Oracle certified (+1:PROCEED)
```

Machine- and AI-Readable Presentation

For the AI Cortex and language server, the identical event is preserved as a structured `RollFrame` or `CognitiveNote`:

```
RollFrame {
    tick_micros: 2100,
    roll_index: 1,
    total_rolls: 22,
    verb: "INSCRIBE".to_string(),
```

```
    dual_rail_in:  (0x0000_0000_0000_0000, 0x0000_0000_0000_0000),
    dual_rail_out: (0x0000_0000_0000_0001, 0x0000_0000_0000_0000),
    quad_state: QuadState::Proceed,
    failpoint_mask: 0x0000_0000,
    intent_aside: Some("Genesis roll initialization".to_string()),
}
```

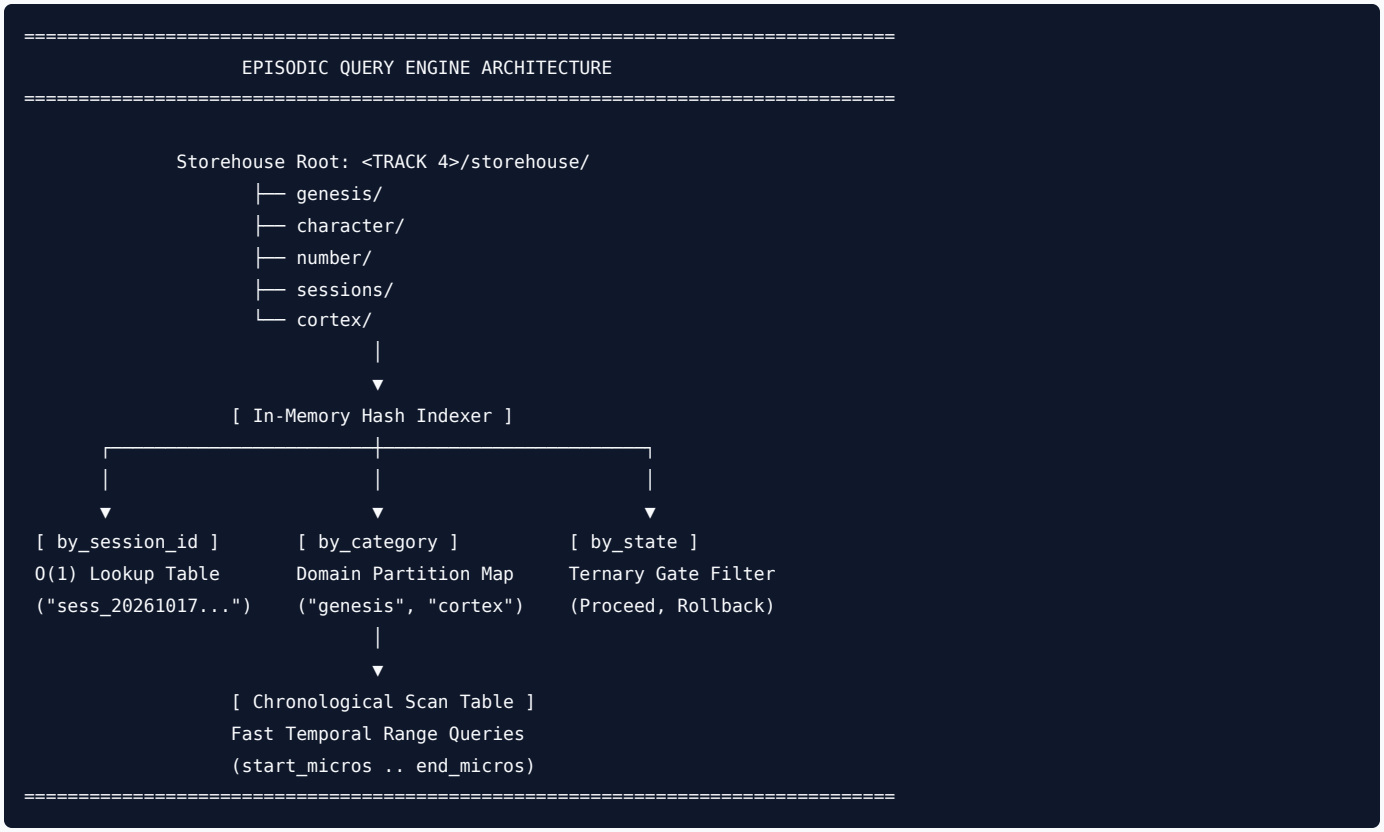
And as a canonical OmniCode `.scroll` note:

```
Note < CognitiveTrace > {
    tick: < 1405 µs >,
    category: [ GENESIS_INCEPTION ],
    summary: < "Balanced Ternary ALU Calibration Verified" >,
    details: < "Silicon identity (+1) + (-1) = 0 evaluated true across registers R0-R12" >,
    tags: [ #genesis :: #silicon :: #first_log :: #baseline :: #omega_0 ]
}
```

5. Episodic Querying and O(1) Cognitive Memory Retrieval

As a computer runs, log volumes grow rapidly. Traditional logging frameworks dump megabytes of unstructured text into sprawling log files, forcing developers and AI agents to execute expensive linear regex scans (`grep`) across the filesystem.

The OmniCode `DiagnosticLogger` rejects linear scans. It provides an **Episodic Query Engine (`EpisodicQueryEngine`)** that indexes execution scrolls and cognitive notes into an $O(1)$ memory matrix:



Core Retrieval Invariants

1. **$O(1)$ Session Retrieval:** Look up any past execution reel instantly using its unique session ID or sanitized file stem (`find_by_session`).
2. **Category Partitioning:** Retrieve all episodes belonging to a specific domain (`"genesis"` , `"cortex"` , `"character"` , `"number"`) without reading other partitions (`filter_by_category`).
3. **QuadState Invariant Filtering:** Instant extraction of all failed or rolled-back runs (`filter_by_state(QuadState::Rollback)`) or runs with active failpoints (`filter_by_failpoint(true)`).

4. **Temporal Window Slicing:** Extract all logs between timestamp t_1 and t_2 via binary search on sorted microsecond keys (`range_scan`).
5. **Associative Tag Discovery:** Locate related cognitive notes through semantic tag sets (`#genesis` , `#first_log` , `#omega_0` , `#remediation_ready`).

---

6. The Complete Autonomic Immune Cycle: A Step-by-Step Scenario

To observe the DAR triad operating as a unified system, trace a concrete scenario: a syntax or boundary collision occurring during a process tape step.

```
[Step 1: Collision Trapped]
The parser or VM encounters an unclosed delimiter: `process_tape < 1 ]`.
  |
  ▼
[Step 2: Detect (Logger)]
The Tripwire trips before any panic can occur.
The `DiagnosticLogger` records a `LogEvent` with `FailpointType::BoundaryMismatch`.
It attaches a `FixSuggestion` indicating line 1, column 17, replacing ']' with '>'.
A `CognitiveNote` is emitted into episodic memory tagged `#failpoint` and `#boundary_mismatch`.
  |
  ▼
[Step 3: Assess (Debugger)]
The `OmniDebugger` captures an immutable snapshot of the active environment.
It detects that the state invariant is violated and executes `-1:ROLLBACK`.
The debugger places the `FixSuggestion` onto its `remediation_queue`.
  |
  ▼
[Step 4: Restore (Restorator)]
The `Restorator` dequeues the suggestion.
It executes morphological boundary repair, mutating the AST candidate: `process_tape < 1 >`.
  |
  ▼
[Step 5: Oracle Verification]
Before committing the patch, the Restorator submits the candidate to the Test Oracle.
The Oracle re-lexes, re-parses, and verifies that zero failpoints occur.
Verification returns `Ok(())` (+1:PROCEED).
  |
  ▼
[Step 6: State Re-Certified]
The healed source is committed.
The `DiagnosticLogger` emits `REPAIR_VERIFIED` to the flight recorder.
The AI Cortex records the episode into long-term memory for future few-shot synthesis.
Execution proceeds smoothly without human intervention.
```

---

7. Covenantal Sealing and Invariant Summary

Invariant ID	Subsystem	Invariant Formulation & Covenant Requirement
INV-DAR-01	**Detection**	Every invariant collision must be trapped by a tripwire; zero unhandled panics or crashes permitted.
INV-DAR-02	**Assessment**	An anomalous state must reverse causal momentum (`-1:ROLLBACK`) to a verified snapshot before healing.
INV-DAR-03	**Restoration**	No candidate patch synthesized by the `Restorator` may be committed without passing a Test Oracle.
INV-DAR-04	**Genesis**	The `DiagnosticLogger` must ingest Track 0 cold-boot telemetry as primordial baseline Ω_0.
INV-DAR-05	**Equilibrium**	The system must return to baseline Ω_0 prior to hardware shutdown or power-off.
INV-DAR-06	**Dual Read**	All diagnostic telemetry must be human-interpretable (ANSI/ASCII) and machine-actionable (4-Block `.scroll`).

> *"The secret things belong unto the LORD our God: but those things which are revealed belong unto us and to our children for ever, that we may do all the words of this law." — **Deuteronomy 29:29 (KJV)***

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/restorator/mod.rs RS | 667 LINES | 2387 WORDS | 27.8 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Restorator Identity, Self-Healing Immune Invariants & Repair Kinds )
// ================================================================================
//! OmniCode Diagnostics: Restorator & Self-Healing Engine
//!
//! # Purpose & Cybernetic Architecture
//! Autonomous code restoration and healing organ. Diagnoses syntax anomalies,
//! unclosed or mismatched boundaries, missing 4-block architecture scaffolding,
//! and missing standard library imports, applying surgical restorative patches
//! to bring non-operational code into a verified operational state (`+1:PROCEED`).
//!
//! # Self-Healing Pipeline Passes:
//! 1. **Pass 1 (Boundary Physics)**: Closes unclosed delimiters and aligns mismatched pairs.
//! 2. **Pass 2 (4-Block Architecture)**: Ensures canonical order: Metadata -> Opening -> Body -> Closing.
//! 3. **Pass 3 (Dependency Injection)**: Automatically detects required core modules (`Core.Arithmetic`, etc.) and injects imports.
//! 4. **Pass 4 (Conceptual Inference)**: Synthesizes noun declarations with infer operators (`Ball < Entity? >`).
//! 5. **Pass 5 (Round-trip Verification)**: Lints, lexes, parses, and evaluates the candidate to certify operational validity.
//!
//! # Test Oracle Integration:
//! Connects directly to test verification harnesses via [`Restorator::restore_and_verify_with_oracle`],
//! ensuring patches satisfy external invariants and recording verified repair notes in AI memory.

use std::fs;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::linter::{BlockKind, OmniLinter};
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, LogEvent};
use crate::parser::lexer::Lexer;
use crate::parser::parser::Parser;
use crate::runtime::environment::Environment;
use crate::runtime::evaluator::Evaluator;


// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Restoration Structures and Engine Context Initialization )
// ================================================================================
/// Record of an individual restorative patch applied to source code.
#[derive(Debug, Clone)]
pub struct AppliedRepair {
    /// The structural category of the applied repair.
    pub kind: FixKind,
    /// Detailed human and AI-readable explanation of why and how the repair was made.
    pub description: String,
    /// 1-based source line number where the repair took effect.
    pub line: usize,
}

/// Comprehensive outcome report generated by the [`Restorator`].
///
/// Contains original source, restored content, applied repairs, verification status,
/// and any errors encountered during the self-healing cycle.
#[derive(Debug, Clone)]
pub struct RestorationReport {
    /// File path if restoration was performed on disk, or `None` if in-memory.
    pub file_path: Option<String>,
    /// `true` if any modifications or repairs were made to the source text.
    pub was_modified: bool,
    /// Exact unmodified original source code prior to healing passes.
    pub original_content: String,
    /// Healed, patched, and structurally validated source code.
    pub restored_content: String,
    /// Chronological list of all automated repairs applied during the cycle.
    pub repairs_applied: Vec<AppliedRepair>,
    /// `true` if the restored content passed all lint, parse, and evaluation checks.
    pub verified: bool,
    /// The final QuadState produced by evaluating the restored source (+1:PROCEED).
    pub final_state: Option<QuadState>,
    /// Any lint, parse, or evaluation errors if verification failed.
    pub errors: Vec<String>,
}
```

```rust

/// The autonomous self-healing and code restoration engine.
pub struct Restorator {
    /// Shared diagnostic logger recording cognitive observations and repair history.
    pub logger: DiagnosticLogger,
}

impl Restorator {
    /// Constructs a new [`Restorator`] instance bound to the provided logger.
    pub fn new(logger: DiagnosticLogger) -> Self {
        Self { logger }
    }
}


// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Restoration Pipeline: Boundaries, 4-Block Headers, Imports, Verification )
// ============================================================================
impl Restorator {
    /// Restores source code in memory, executing multi-pass auto-repairs.
    ///
    /// Executes the 5-pass self-healing pipeline:
    /// 1. Delimiter & Boundary Healing
    /// 2. 4-Block Architecture Injection
    /// 3. Standard Library Import Resolution
    /// 4. Inferred Conceptual Entity Declaration
    /// 5. Round-trip Lint/Parse/Eval Verification
    ///
    /// # Arguments
    /// * `source` - Raw input string that may contain boundary leaks or syntax anomalies.
    ///
    /// # Returns
    /// A [`RestorationReport`] detailing all modifications and verification proofs.
    pub fn restore_source(&self, source: &str) -> RestorationReport {
        let original_content = source.to_string();
        let mut current = source.to_string();
        let mut repairs_applied = Vec::new();

        // Pass 1: Delimiter & Boundary Healing
        current = self.heal_boundaries(&current, &mut repairs_applied);

        // Pass 2: 4-Block Architectural Scaffolding Healing
        current = self.heal_four_blocks(&current, &mut repairs_applied);

        // Pass 3: Automated Dependency & Import Healing
        current = self.heal_imports(&current, &mut repairs_applied);

        // Pass 4: Inferred Conceptual Entity Healing
        current = self.heal_inferred_entities(&current, &mut repairs_applied);

        // Pass 5: Round-trip Self-Verification
        let (verified, final_state, errors) = self.verify_restoration(&current);

        let was_modified = current != original_content;

        if !repairs_applied.is_empty() {
            let repair_descs: Vec<String> = repairs_applied.iter().map(|r| r.description.clone()).collect();
            self.logger.record_ai_observation(
                "SELF_HEAL_APPLIED",
                &format!("Applied {} automated repairs", repairs_applied.len()),
                &format!("Repairs executed: {}", repair_descs.join("; ")),
                ["self_healing", "remediation", "restorator"].as_slice(),
                None,
            );
        }

        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
    }
```

```rust

    /// Ingests a structured [`LogEvent`] and executes a targeted, single-point surgical patch.
    ///
    /// # Arguments
    /// * `source` - Source code to repair.
    /// * `log` - Structured incident event containing a [`crate::diagnostics::logger::FixSuggestion`].
    ///
    /// # Returns
    /// [`RestorationReport`] showing the single-point modification and round-trip verification status.
    pub fn restore_from_incident_log(&self, source: &str, log: &LogEvent) -> RestorationReport {
        let original_content = source.to_string();
        let mut repairs_applied = Vec::new();
        let mut current = source.to_string();

        if let Some(sug) = &log.suggestion {
            match sug.kind {
                FixKind::FixBoundaryMismatch => {
                    let lines: Vec<&str> = current.lines().collect();
                    if sug.line > 0 && sug.line <= lines.len() {
                        let mut target_line = lines[sug.line - 1].to_string();
                        if let Some(pos) = target_line.find(&sug.target_token) {
                            target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                            let mut new_lines = lines.clone();
                            new_lines[sug.line - 1] = &target_line;
                            current = new_lines.join("\n");
                            if !current.ends_with('\n') {
                                current.push('\n');
                            }
                            repairs_applied.push(AppliedRepair {
                                kind: FixKind::FixBoundaryMismatch,
                                description: sug.rationale.clone(),
                                line: sug.line,
                            });
                        }
                    }
                }
                FixKind::CloseBoundary => {
                    current.push_str(&sug.suggested_patch);
                    if !current.ends_with('\n') {
                        current.push('\n');
                    }
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::CloseBoundary,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                FixKind::ConvertImmutableToMutable => {
                    current = current.replace(&format!("< {} >", sug.target_token), &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: FixKind::ConvertImmutableToMutable,
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
                _ => {
                    current = current.replace(&sug.target_token, &sug.suggested_patch);
                    repairs_applied.push(AppliedRepair {
                        kind: sug.kind.clone(),
                        description: sug.rationale.clone(),
                        line: sug.line,
                    });
                }
            }
        }

        let (verified, final_state, errors) = self.verify_restoration(&current);
        let was_modified = current != original_content;

        if was_modified {
            self.logger.record_ai_observation(
                "LOG_DRIVEN_RESTORE",
                &format!("Applied targeted fix from log for {:?}", log.failpoint_type),
                &format!("Modified {} bytes", current.len()),
                ["self_healing", "log_driven"].as_slice(),
                None,
            );
        }
```

```
        RestorationReport {
            file_path: None,
            was_modified,
            original_content,
            restored_content: current,
            repairs_applied,
            verified,
            final_state,
            errors,
        }
    }

    /// Restores source code and executes an external verification test oracle against the candidate.
    ///
    /// Connects the self-healing engine to test harnesses:
    /// - If the restored code satisfies the test oracle: logs an `ORACLE_CERTIFIED` cognitive trace.
    /// - If the test oracle fails: records a rollback failpoint and adds the failure to the report errors.
    ///
    /// # Arguments
    /// * `source` - Raw input code.
    /// * `test_oracle` - Closure asserting domain-specific invariants on the restored source.
    pub fn restore_and_verify_with_oracle<F>(&self, source: &str, test_oracle: F) -> RestorationReport
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let mut report = self.restore_source(source);

        if report.verified {
            match test_oracle(&report.restored_content) {
                Ok(()) => {
                    self.logger.log("RESTORATOR", "Restoration validated by test oracle (+1:PROCEED)");
                    self.logger.record_ai_observation(
                        "ORACLE_CERTIFIED",
                        "Restored code certified by test oracle",
                        "The candidate passed the verification test suite cleanly",
                        ["self_healing", "test_oracle", "certified"].as_slice(),
                        None,
                    );
                }
                Err(err) => {
                    self.logger.failpoint(
                        FailpointType::RollbackTriggered,
                        "RESTORATOR",
                        format!("Restoration failed test oracle verification: {err}"),
                    );
                    self.logger.record_ai_observation(
                        "ORACLE_REJECTED",
                        "Restored code failed test oracle verification",
                        &err,
                        ["self_healing", "test_oracle", "rollback"].as_slice(),
                        None,
                    );
                    report.verified = false;
                    report.errors.push(format!("Test oracle failure: {err}"));
                }
            }
        }

        report
    }

    /// Restores a file from disk, optionally writing back the healed source in-place.
    ///
    /// # Arguments
    /// * `path` - File system path to the target source file.
    /// * `in_place` - If true, overwrites the file on disk with the restored content.
    ///
    /// # Errors
    /// Returns `Err(String)` if file reading or writing fails.
    pub fn restore_file(&self, path: &str, in_place: bool) -> Result<RestorationReport, String> {
        let content = fs::read_to_string(path)
            .map_err(|e| format!("Failed to read file '{path}': {e}"))?;

        let mut report = self.restore_source(&content);
        report.file_path = Some(path.to_string());

        if in_place && report.was_modified {
```

```
            fs::write(path, &report.restored_content)
                .map_err(|e| format!("Failed to write restored file '{path}': {e}"))?;
            self.logger.log("RESTORATOR", format!("Wrote restored content in-place to '{path}'"));
        }

        Ok(report)
    }

    /// Pass 1: Auto-detects and repairs mismatched and leaking delimiters.
    fn heal_boundaries(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut text = source.to_string();
        let mut max_passes = 10;

        while max_passes > 0 {
            max_passes -= 1;
            let temp_logger = DiagnosticLogger::new(false);
            let mut lexer = Lexer::new(&text, temp_logger.clone());
            let tokens = match lexer.tokenize() {
                Ok(toks) => toks,
                Err(_) => break,
            };

            let debugger = OmniDebugger::new(temp_logger.clone(), 10);
            let mut parser = Parser::new(tokens, debugger);
            let parse_res = parser.parse_program();

            if parse_res.is_ok() {
                break; // Boundaries are sound
            }

            // Inspect diagnostic remediations from debugger
            let mut debugger_remediations = parser.debugger.drain_remediations();
            if debugger_remediations.is_empty() {
                debugger_remediations = temp_logger.remediations();
            }

            if debugger_remediations.is_empty() {
                // If EOF unclosed boundary detected without explicit remediation, balance stack
                if !parser.debugger.boundary_stack.is_empty() {
                    let mut closers = String::new();
                    while let Some(frame) = parser.debugger.boundary_stack.pop() {
                        let c = match frame.open_char {
                            '<' => '>',
                            '[' => ']',
                            '{' => '}',
                            _ => ')',
                        };
                        closers.push(c);
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: format!("Appended missing closing delimiter '{c}' for boundary opened at
L{}:C{}", frame.line, frame.col),
                            line: frame.line,
                        });
                    }
                    text.push_str(&format!("\n{}\n", closers));
                    continue;
                }
                break;
            }

            // Apply remediation patches
            let mut applied_any = false;
            for sug in debugger_remediations {
                match sug.kind {
                    FixKind::FixBoundaryMismatch => {
                        let lines: Vec<&str> = text.lines().collect();
                        if sug.line > 0 && sug.line <= lines.len() {
                            let mut target_line = lines[sug.line - 1].to_string();
                            if let Some(pos) = target_line.find(&sug.target_token) {
                                target_line.replace_range(pos..pos + sug.target_token.len(), &sug.suggested_patch);
                                let mut new_lines = lines.clone();
                                new_lines[sug.line - 1] = &target_line;
                                text = new_lines.join("\n");
                                if !text.ends_with('\n') {
                                    text.push('\n');
                                }
                                repairs.push(AppliedRepair {
```

```
                                    kind: FixKind::FixBoundaryMismatch,
                                    description: sug.rationale,
                                    line: sug.line,
                                });
                                applied_any = true;
                                break;
                            }
                        }
                    }
                    FixKind::CloseBoundary => {
                        text.push_str(&sug.suggested_patch);
                        if !text.ends_with('\n') {
                            text.push('\n');
                        }
                        repairs.push(AppliedRepair {
                            kind: FixKind::CloseBoundary,
                            description: sug.rationale,
                            line: sug.line,
                        });
                        applied_any = true;
                        break;
                    }
                    _ => {}
                }
            }

            if !applied_any {
                break;
            }
        }

        text
    }

    /// Pass 2: Ensures all 4 universal architectural blocks exist in canonical order.
    fn heal_four_blocks(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let linter = OmniLinter::new(DiagnosticLogger::new(false));
        let report = linter.lint_source("memory", source);

        if report.is_valid {
            return source.to_string();
        }

        let mut has_metadata = false;
        let mut has_opening = false;
        let mut has_body = false;
        let mut has_closing = false;

        for b in &report.blocks_found {
            match b.kind {
                BlockKind::Metadata => has_metadata = true,
                BlockKind::Opening => has_opening = true,
                BlockKind::Body => has_body = true,
                BlockKind::Closing => has_closing = true,
            }
        }

        let mut out = String::new();

        // 1. Inject Metadata Block if missing
        if !has_metadata {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 1: METADATA manifest".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 1: METADATA BLOCK\n");
            out.push_str("# ( Restored Module Manifest and Semantic Track )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Module < Restored.Track > ( Auto-restored OmniCode compilation unit ) {\n");
            out.push_str("    version: < \"1.0.0\" >,\n");
            out.push_str("    system: < OmniCode.Core >\n");
            out.push_str("}\n\n");
        }

        // 2. Inject Opening Block if missing
        if !has_opening {
```

```
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 2: OPENING context".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 2: OPENING BLOCK\n");
            out.push_str("# ( Initial Conditions and Operational Preconditions )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Opening {\n");
            out.push_str("    preconditions: [ +1 ]\n");
            out.push_str("}\n\n");
        }

        // 3. Append existing body content
        if !has_body {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 3: BODY block wrapping loose operations".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 3: BODY BLOCK\n");
            out.push_str("# ( Core Operations and Transformations )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Body [\n");
            for line in source.lines() {
                if !line.trim().is_empty() {
                    out.push_str(&format!("    {}\n", line));
                }
            }
            out.push_str("]\n\n");
        } else {
            out.push_str(source);
            if !out.ends_with('\n') {
                out.push('\n');
            }
        }

        // 4. Inject Closing Block if missing
        if !has_closing {
            repairs.push(AppliedRepair {
                kind: FixKind::AddBlockHeader,
                description: "Injected canonical BLOCK 4: CLOSING commit proof".into(),
                line: 1,
            });
            out.push_str("# ================================================================================\n");
            out.push_str("# BLOCK 4: CLOSING BLOCK\n");
            out.push_str("# ( State Verification Proof and Final Commit )\n");
            out.push_str("# ================================================================================\n");
            out.push_str("Closing {\n");
            out.push_str("    commit_status [ PROCEED ]\n");
            out.push_str("}\n");
        }

        out
    }

    /// Pass 3: Automatically injects missing core library imports into Opening block.
    fn heal_imports(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let needs_arithmetic = (source.contains("plus") || source.contains("minus") || source.contains("times") ||
source.contains("div") || source.contains("ADD") || source.contains("SUB"))
            && !source.contains("Core.Arithmetic");

        let needs_logic = (source.contains("PROCEED") || source.contains("ROLLBACK") || source.contains("AWAIT") ||
source.contains("HALT"))
            && !source.contains("Core.Logic");

        let needs_vectors = (source.contains("-->") || source.contains("<--") || source.contains("--^") ||
source.contains("--v"))
            && !source.contains("Core.Vectors");

        if !needs_arithmetic && !needs_logic && !needs_vectors {
            return source.to_string();
        }

        let mut mods_to_import = Vec::new();
        if needs_arithmetic {
```

```
            mods_to_import.push("Core.Arithmetic");
        }
        if needs_logic {
            mods_to_import.push("Core.Logic");
        }
        if needs_vectors {
            mods_to_import.push("Core.Vectors");
        }

        let import_expr = format!("import: [ {} ]", mods_to_import.join(" :: "));

        // Inject into Opening { ... } block
        if let Some(op_idx) = source.find("Opening {") {
            let insert_pos = op_idx + "Opening {".len();
            let mut out = source.to_string();
            let injection = format!("\n    {},\n", import_expr);
            out.insert_str(insert_pos, &injection);

            repairs.push(AppliedRepair {
                kind: FixKind::AddImport,
                description: format!("Injected library imports into Opening block: {import_expr}"),
                line: 1,
            });
            return out;
        }

        source.to_string()
    }

    /// Pass 4: Resolves and declares undefined conceptual nouns with infer operator `?`.
    fn heal_inferred_entities(&self, source: &str, repairs: &mut Vec<AppliedRepair>) -> String {
        let mut missing_inferred = Vec::new();

        if source.contains("Ball?") && !source.contains("Ball <") {
            missing_inferred.push("Ball < Entity? > ( Inferred discrete unit )");
        }
        if source.contains("Group?") && !source.contains("Group <") {
            missing_inferred.push("Group < Set? >   ( Inferred collection container )");
        }

        if missing_inferred.is_empty() {
            return source.to_string();
        }

        let mut out = source.to_string();
        // Insert right after Module < ... > definition
        if let Some(pos) = out.find("Module <") {
            if let Some(block_end) = out[pos..].find('}') {
                let insert_at = pos + block_end + 1;
                let mut decls = String::from("\n\n# Auto-Restored Conceptual Types:\n");
                for d in &missing_inferred {
                    decls.push_str(d);
                    decls.push('\n');
                }
                out.insert_str(insert_at, &decls);

                repairs.push(AppliedRepair {
                    kind: FixKind::DeclareEntityWithInfer,
                    description: format!("Declared {} inferred conceptual entities", missing_inferred.len()),
                    line: 1,
                });
            }
        }

        out
    }

    /// Pass 5: Evaluates restored code to verify complete execution without failpoints.
    fn verify_restoration(&self, source: &str) -> (bool, Option<QuadState>, Vec<String>) {
        let mut errors = Vec::new();
        let logger = DiagnosticLogger::new(false);

        // 1. Lint check
        let linter = OmniLinter::new(logger.clone());
        let lint_rep = linter.lint_source("verify", source);
        if !lint_rep.is_valid {
            for err in lint_rep.errors {
                errors.push(format!("Lint error: {err}"));
```

```
            }
        }

        // 2. Tokenize & Parse
        let mut lexer = Lexer::new(source, logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(t) => t,
            Err(e) => {
                errors.push(format!("Lex error: {e}"));
                return (false, None, errors);
            }
        };

        let debugger = OmniDebugger::new(logger.clone(), 10);
        let mut parser = Parser::new(tokens, debugger);
        let prog = match parser.parse_program() {
            Ok(p) => p,
            Err(e) => {
                errors.push(format!("Parse error: {e}"));
                return (false, None, errors);
            }
        };

        // 3. Execution check
        let mut evaluator = Evaluator::new(Environment::new(), parser.debugger);
        match evaluator.eval(&crate::ast::nodes::ASTNode::Program(prog)) {
            Ok(val) => {
                let state = match val {
                    crate::runtime::environment::RuntimeValue::State(s) => s,
                    _ => QuadState::Proceed,
                };
                let verified = errors.is_empty() && (state == QuadState::Proceed || state == QuadState::Await);
                (verified, Some(state), errors)
            }
            Err(e) => {
                errors.push(format!("Eval error: {e}"));
                (false, None, errors)
            }
        }
    }
}


// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Restorator Verification and Default Sealing )
// ================================================================================
impl Default for Restorator {
    fn default() -> Self {
        Self::new(DiagnosticLogger::new(false))
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/debugger/mod.rs RS 471 LINES 1835 WORDS 20.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniDebugger Identity, Invariant Verification Engine & Safety Constraints )
// ============================================================================
//! OmniCode Diagnostics: Debugger & Invariant Inspector
//!
//! # Purpose & Cybernetic Architecture
//! Acts as the runtime invariant enforcement organ and rollback controller:
//! - **Boundary Physics**: Manages the active delimiter stack, trapping leaks and mismatches.
//! - **Immutability Protection**: Enforces Rank 1 axiom protection `< ... >` against illegal mutation.
//! - **Retroaction Bounding**: Prevents infinite rollback loops via threshold counters.
//! - **Self-Healing Test Oracles**: Connects candidate restorations directly to test verification
//!   harnesses, certifying patches before committing state (+1:PROCEED).
//!
//! # IDE Hover & AI Agent Context
//! When hovering over [`OmniDebugger`] or its methods in an IDE:
//! - Hovering over [`OmniDebugger::execute_verification_test`] explains how verification sandboxes work.
//! - Hovering over [`OmniDebugger::execute_restoration_from_log`] details log-driven surgical patching.
//! - Hovering over [`OmniDebugger::pop_boundary`] shows delimiter matching and remediation synthesis.

use std::collections::{HashMap, HashSet};
use crate::core::boundaries::BoundaryType;
use crate::core::ternary::QuadState;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion, LogEvent};


// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( BoundaryFrame Definition and OmniDebugger Context Setup )
// ============================================================================
/// Stack frame recording an unclosed boundary delimiter's opening context.
#[derive(Debug, Clone)]
pub struct BoundaryFrame {
    /// The opening delimiter character: `<`, `[`, `{`, or `(`.
    pub open_char: char,
    /// 1-based source line where the boundary opened.
    pub line: usize,
    /// 1-based source column where the delimiter occurred.
    pub col: usize,
}

/// Runtime invariant inspector, boundary stack tracker, and test verification oracle.
#[derive(Debug, Clone)]
pub struct OmniDebugger {
    /// Shared diagnostic logger for emitting debug events, failpoints, and cognitive traces.
    pub logger: DiagnosticLogger,
    /// Active LIFO stack of unclosed boundary frames.
    pub boundary_stack: Vec<BoundaryFrame>,
    /// Set of entity names declared inside `< ... >` sealed against mutation.
    pub immutable_registry: HashSet<String>,
    /// Execution rollback counters keyed by target label to detect infinite retroaction loops.
    pub retroaction_counters: HashMap<String, usize>,
    /// Staged queue of automated remediation suggestions awaiting application by the Restorator.
    pub remediation_queue: Vec<FixSuggestion>,
    /// Maximum permitted rollback operations per target before triggering cyclic retroaction failure.
    pub max_retroaction_depth: usize,
    /// Total operational steps and state evaluations inspected by this debugger.
    pub step_count: usize,
}

impl OmniDebugger {
    /// Constructs a new [`OmniDebugger`].
    ///
    /// # Arguments
    /// * `logger` - The shared [`DiagnosticLogger`] instance for telemetry recording.
    /// * `max_retroaction_depth` - Threshold ceiling for retroactions before trapping cyclic loops.
    pub fn new(logger: DiagnosticLogger, max_retroaction_depth: usize) -> Self {
        Self {
            logger,
            boundary_stack: Vec::new(),
            immutable_registry: HashSet::new(),
            retroaction_counters: HashMap::new(),
            remediation_queue: Vec::new(),
            max_retroaction_depth,
```

```
            step_count: 0,
        }
    }
}


// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Core Invariant Enforcement: Boundary Push/Pop, Immutability, Retroaction )
// ================================================================================
impl OmniDebugger {
    /// Pushes an opening boundary delimiter onto the active stack.
    ///
    /// # Arguments
    /// * `open` - Delimiter character (`<`, `[`, `{`, `(`).
    /// * `line` - 1-based line number.
    /// * `col` - 1-based column number.
    pub fn push_boundary(&mut self, open: char, line: usize, col: usize) {
        self.logger.log(
            "BOUNDARY",
            format!("Pushing delimiter '{open}' at L{line}:C{col} (depth: {})", self.boundary_stack.len() + 1),
        );
        self.boundary_stack.push(BoundaryFrame { open_char: open, line, col });
    }

    /// Pops a closing boundary delimiter and validates the pair against the stack top.
    ///
    /// # Failpoints Intercepted
    /// - **Empty Stack**: Traps unmatched closing delimiter as [`FailpointType::BoundaryMismatch`].
    /// - **Mismatched Delimiter**: Traps illegal pair crossing (e.g. `< ... ]`), logs a
    ///   [`FixSuggestion`] pointing to the correct partner, and queues remediation.
    ///
    /// # Returns
    /// `Ok(BoundaryType)` on valid resolution, or `Err(String)` on failpoint mismatch.
    pub fn pop_boundary(&mut self, close: char, line: usize, col: usize) -> Result<BoundaryType, String> {
        if self.boundary_stack.is_empty() {
            let msg = format!("Unmatched closing delimiter '{close}' at L{line}:C{col} (stack is empty)");
            self.logger.failpoint(FailpointType::BoundaryMismatch, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        let frame = self.boundary_stack.pop().unwrap();
        match BoundaryType::from_delimiters(frame.open_char, close) {
            Ok(btype) => {
                self.logger.log(
                    "BOUNDARY",
                    format!(
                        "Successfully resolved boundary {} '{}{}' (remaining depth: {})",
                        btype, frame.open_char, close, self.boundary_stack.len()
                    ),
                );
                Ok(btype)
            }
            Err(err) => {
                let msg = format!(
                    "Boundary mismatch: opened with '{}' (L{}:C{}) but closed with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col, close, line, col
                );
                let matching = match frame.open_char {
                    '<' => '>',
                    '[' => ']',
                    '{' => '}',
                    _ => ')',
                };
                let suggestion = FixSuggestion {
                    kind: FixKind::FixBoundaryMismatch,
                    line,
                    col,
                    target_token: close.to_string(),
                    suggested_patch: matching.to_string(),
                    rationale: format!(
                        "Replace mismatched closing delimiter '{close}' with matching '{matching}' to seal boundary
opened with '{}' at L{}:C{}",
                        frame.open_char, frame.line, frame.col
                    ),
                };
                self.logger.failpoint_with_remediation(
                    FailpointType::BoundaryMismatch,
```

```
                    "DEBUGGER",
                    &msg,
                    suggestion.clone(),
                );
                self.remediation_queue.push(suggestion);
                Err(err)
            }
        }
    }

    /// Asserts that no boundaries remain unclosed at EOF.
    ///
    /// If unclosed delimiters remain on the stack, logs [`FailpointType::BoundaryLeak`]
    /// with a concrete [`FixSuggestion`] indicating the required closing tokens.
    pub fn assert_all_boundaries_closed(&mut self) -> Result<(), String> {
        if let Some(frame) = self.boundary_stack.last() {
            let msg = format!(
                "Boundary leak: delimiter '{}' opened at L{}:C{} was never closed",
                frame.open_char, frame.line, frame.col
            );
            let matching = match frame.open_char {
                '<' => '>',
                '[' => ']',
                '{' => '}',
                _ => ')',
            };
            let suggestion = FixSuggestion {
                kind: FixKind::CloseBoundary,
                line: frame.line,
                col: frame.col,
                target_token: frame.open_char.to_string(),
                suggested_patch: format!("\n{matching}"),
                rationale: format!(
                    "Append missing closing delimiter '{matching}' for boundary opened with '{}' at L{}:C{}",
                    frame.open_char, frame.line, frame.col
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::BoundaryLeak,
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Registers an entity identifier defined inside `< ... >` as globally immutable Rank 1 axiom.
    pub fn register_immutable(&mut self, name: &str) {
        self.immutable_registry.insert(name.to_string());
        self.logger.log("IMMUTABILITY", format!("Registered invariant <{name}>"));
    }

    /// Verifies that mutation is permitted for the given entity identifier.
    ///
    /// # Failpoints Intercepted
    /// If the entity was registered as immutable, traps [`FailpointType::ImmutableMutation`],
    /// and suggests converting the declaration to a mutable Rank 2 denotation `[ ... ]`.
    pub fn assert_mutation_permitted(&mut self, name: &str) -> Result<(), String> {
        if self.immutable_registry.contains(name) {
            let msg = format!(
                "Illegal mutation attempted on immutable invariant <{name}>! Entity cannot be changed."
            );
            let suggestion = FixSuggestion {
                kind: FixKind::ConvertImmutableToMutable,
                line: 0,
                col: 0,
                target_token: name.to_string(),
                suggested_patch: format!("[ {name} ]"),
                rationale: format!(
                    "Entity <{name}> was sealed as a Rank 1 immutable invariant; declare it with '[ {name} ]' if updates
are required"
                ),
            };
            self.logger.failpoint_with_remediation(
                FailpointType::ImmutableMutation,
```

```
                "DEBUGGER",
                &msg,
                suggestion.clone(),
            );
            self.remediation_queue.push(suggestion);
            return Err(format!("FAILPOINT: {msg}"));
        }
        Ok(())
    }

    /// Retrieves and empties the active remediation queue.
    pub fn drain_remediations(&mut self) -> Vec<FixSuggestion> {
        std::mem::take(&mut self.remediation_queue)
    }

    /// Monitors retroaction steps on a named target to prevent infinite rollback loops.
    ///
    /// # Failpoints Intercepted
    /// If rollback iterations exceed `max_retroaction_depth`, traps [`FailpointType::CyclicRetroaction`].
    pub fn track_retroaction(&mut self, target: &str) -> Result<usize, String> {
        let count = self.retroaction_counters.entry(target.to_string()).or_insert(0);
        *count += 1;
        let cur = *count;

        self.logger.log(
            "VECTOR",
            format!("Retroaction step #{cur} on '{target}'"),
        );

        if cur > self.max_retroaction_depth {
            let msg = format!(
                "Cyclic retroaction loop detected! Target '{target}' rolled back {cur} times, \
                 exceeding threshold ({})",
                self.max_retroaction_depth
            );
            self.logger.failpoint(FailpointType::CyclicRetroaction, "DEBUGGER", &msg);
            return Err(format!("FAILPOINT: {msg}"));
        }

        Ok(cur)
    }

    /// Inspects QuadState and raises appropriate signals or failpoints.
    ///
    /// - `QuadState::Halt`: Traps void / NULL halt trap (`FP_NULL_TRAP`).
    /// - `QuadState::Rollback`: Traps rollback trigger (`FP_ROLLBACK_TRIGGERED`).
    /// - `QuadState::Await`: Standby state logged.
    /// - `QuadState::Proceed`: Commit state logged.
    pub fn inspect_state(&mut self, state: QuadState, context: &str) {
        self.step_count += 1;
        match state {
            QuadState::Halt => {
                self.logger.failpoint(
                    FailpointType::NullTrap,
                    "DEBUGGER",
                    format!("NULL / Void halt trap encountered in context: {context}"),
                );
            }
            QuadState::Rollback => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "DEBUGGER",
                    format!("Rollback (-1) triggered in context: {context}"),
                );
            }
            QuadState::Await => {
                self.logger.log("TERNARY", format!("Await (0) standby in context: {context}"));
            }
            QuadState::Proceed => {
                self.logger.log("TERNARY", format!("Proceed (+1) committed in context: {context}"));
            }
        }
    }

    /// Executes a verification test against an isolated child debugger context.
    ///
    /// # Why Test Oracles Matter for Self-Healing
    /// Acts as an execution oracle for hypothesis verification before state commit.
```

```rust
    /// The debugger executes the candidate logic in a sandbox:
    /// - If the test passes: records an `ORACLE_SUCCESS` observation for AI memory.
    /// - If the test fails: traps a rollback failpoint and logs `ORACLE_FAILURE`.
    ///
    /// # Arguments
    /// * `test_name` - Human and AI readable identifier for the verification assertion.
    /// * `test_runner` - Closure executing the test logic against a sandboxed child debugger.
    pub fn execute_verification_test<F>(&mut self, test_name: &str, test_runner: F) -> Result<(), String>
    where
        F: FnOnce(&mut OmniDebugger) -> Result<(), String>,
    {
        self.logger.log("TEST_ORACLE", format!("Spawning verification test: {test_name}"));
        let mut test_debugger = OmniDebugger::new(self.logger.clone(), self.max_retroaction_depth);
        match test_runner(&mut test_debugger) {
            Ok(()) => {
                self.logger.log("TEST_ORACLE", format!("Verification test '{test_name}' passed (+1:PROCEED)"));
                self.logger.record_ai_observation(
                    "ORACLE_SUCCESS",
                    &format!("Test '{test_name}' passed"),
                    "Candidate state passed invariant verification without failpoints",
                    ["test_oracle", "verified", "proceed"].as_slice(),
                    Some(test_name),
                );
                Ok(())
            }
            Err(err) => {
                self.logger.failpoint(
                    FailpointType::RollbackTriggered,
                    "TEST_ORACLE",
                    format!("Verification test '{test_name}' failed: {err}"),
                );
                self.logger.record_ai_observation(
                    "ORACLE_FAILURE",
                    &format!("Test '{test_name}' rejected candidate"),
                    &err,
                    ["test_oracle", "rejected", "rollback"].as_slice(),
                    Some(test_name),
                );
                Err(err)
            }
        }
    }

    /// Verifies candidate source code with a provided test oracle function.
    ///
    /// # Arguments
    /// * `candidate_source` - The restored source string under evaluation.
    /// * `test_oracle` - Test function verifying candidate execution invariants.
    ///
    /// # Returns
    /// - `Ok(String)`: If the oracle passes, returns the confirmed candidate source.
    /// - `Err(String)`: If the oracle fails, triggers a state rollback (`-1:ROLLBACK`) and returns the error.
    pub fn verify_candidate_repair<F>(
        &mut self,
        candidate_source: &str,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        self.logger.log("SELF_HEAL", "Verifying candidate repair via test oracle...");
        match test_oracle(candidate_source) {
            Ok(()) => {
                self.logger.log("SELF_HEAL", "Candidate repair verified (+1:PROCEED)!");
                self.inspect_state(QuadState::Proceed, "candidate_repair_verified");
                self.logger.record_ai_observation(
                    "REPAIR_VERIFIED",
                    "Candidate source passed verification oracle",
                    "The proposed restorative patch executed cleanly without failpoints",
                    ["self_healing", "oracle_passed", "verified"].as_slice(),
                    None,
                );
                Ok(candidate_source.to_string())
            }
            Err(e) => {
                self.inspect_state(QuadState::Rollback, "candidate_repair_rejected");
                let msg = format!("Candidate repair failed test verification: {e}");
                self.logger.failpoint(FailpointType::RollbackTriggered, "SELF_HEAL", &msg);
```

```
                self.logger.record_ai_observation(
                    "REPAIR_REJECTED",
                    "Candidate source failed verification oracle",
                    &msg,
                    ["self_healing", "oracle_failed", "rollback"].as_slice(),
                    None,
                );
                Err(msg)
            }
        }
    }

    /// Ingests a structured [`LogEvent`] containing a [`FixSuggestion`], applies the patch,
    /// and invokes the verification oracle before committing the restored code.
    ///
    /// # Arguments
    /// * `source` - The original broken source code.
    /// * `log_event` - The structured incident log emitted by the [`DiagnosticLogger`].
    /// * `test_oracle` - The verification test oracle closure.
    ///
    /// # Returns
    /// `Ok(healed_source)` if the patch was applied and certified by the oracle.
    pub fn execute_restoration_from_log<F>(
        &mut self,
        source: &str,
        log_event: &LogEvent,
        test_oracle: F,
    ) -> Result<String, String>
    where
        F: Fn(&str) -> Result<(), String>,
    {
        let suggestion = log_event.suggestion.as_ref().ok_or_else(|| {
            "LogEvent does not contain remediation suggestion for automated restoration".to_string()
        })?;

        self.logger.log(
            "SELF_HEAL",
            format!("Applying remediation suggestion from log: {:?}", suggestion.kind),
        );

        let mut candidate = source.to_string();
        match suggestion.kind {
            FixKind::FixBoundaryMismatch => {
                let lines: Vec<&str> = candidate.lines().collect();
                if suggestion.line > 0 && suggestion.line <= lines.len() {
                    let mut target_line = lines[suggestion.line - 1].to_string();
                    if let Some(pos) = target_line.find(&suggestion.target_token) {
                        target_line.replace_range(pos..pos + suggestion.target_token.len(),
&suggestion.suggested_patch);
                        let mut new_lines = lines.clone();
                        new_lines[suggestion.line - 1] = &target_line;
                        candidate = new_lines.join("\n");
                        if !candidate.ends_with('\n') {
                            candidate.push('\n');
                        }
                    }
                }
            }
            FixKind::CloseBoundary => {
                candidate.push_str(&suggestion.suggested_patch);
                if !candidate.ends_with('\n') {
                    candidate.push('\n');
                }
            }
            FixKind::ConvertImmutableToMutable => {
                candidate = candidate.replace(
                    &format!("< {} >", suggestion.target_token),
                    &suggestion.suggested_patch,
                );
            }
            _ => {
                candidate = candidate.replace(&suggestion.target_token, &suggestion.suggested_patch);
            }
        }

        // Call the test oracle to certify candidate
        self.verify_candidate_repair(&candidate, test_oracle)
    }
```

```
}


// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Postcondition Verification and Debugger Status )
// ============================================================================
// OmniDebugger safety invariants verified and closed.

```